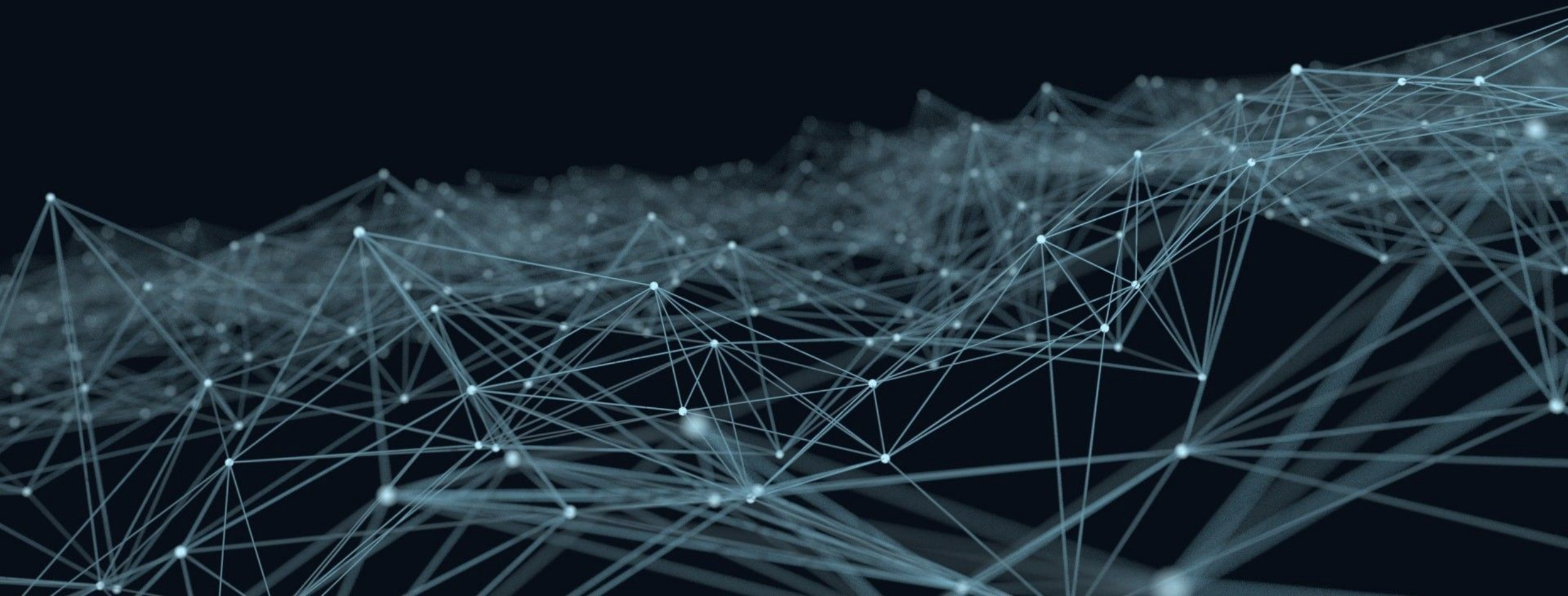
Java Full Stack Development

1. Scanning



TEKsystems Global Services

WORKFORCE DEVELOPMENT

Module Topics

- Static Security Application Testing
- Dynamic Security Application Testing

Static Security Application Testing

SAST

- Static Application Security Testing
  - Static
    - Analyzes source code or compiled bytecode without executing it
  - Application
    - Focuses on the code your team writes, not the OS or infrastructure
  - Security Testing
    - Looks specifically for security defects, not bugs in general
  - Runs early and often
    - Every commit, every pull request, every CI build
    - The "shift left" philosophy
    - Find vulnerabilities at the cheapest point in the SDLC

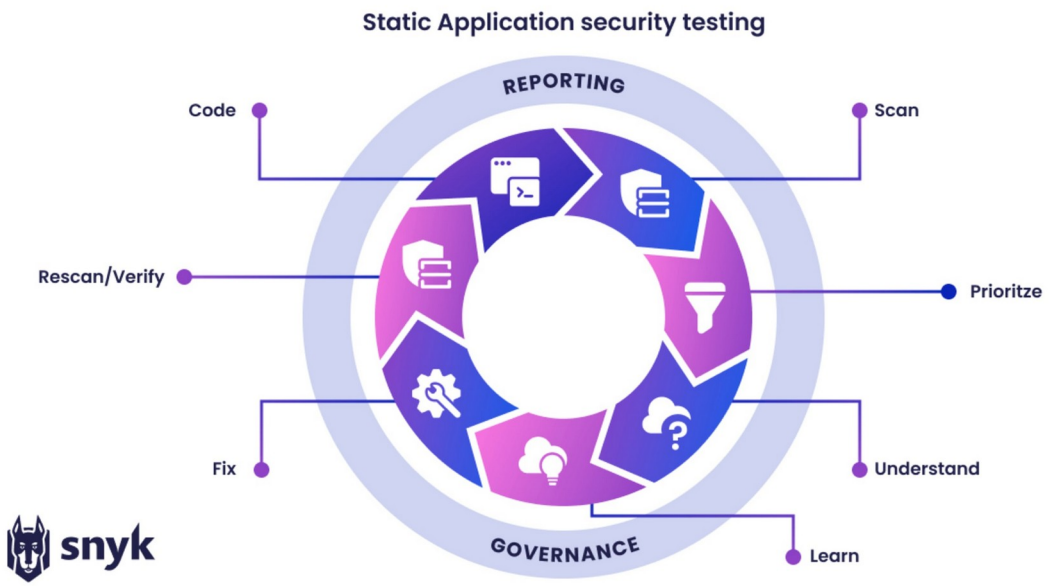


Image Credit:https://snyk.io/fr/articles/application-security/static-application-security-testing/

SAST

- Static
  - Code is analyzed without being runC
  - The analyzer reads the source files
    - Builds a model of what the code does
    - Looks for patterns that indicate vulnerabilities
    - Compare with dynamic analysis (DAST), which runs the application and probes it from outside
    - Static is faster, cheaper, and earlier in the SDLC but dynamic catches things static cannot
- Application
  - For application code
    - Not for operating system vulnerabilities, network misconfigurations, or hardware issues
    - Those are covered by other tools

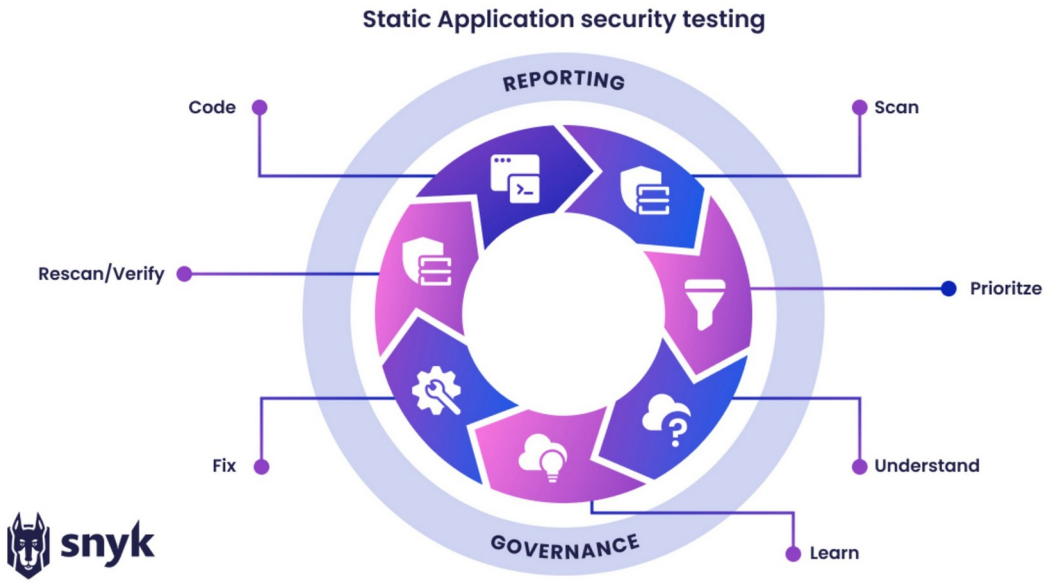


Image Credit:https://snyk.io/fr/articles/application-security/static-application-security-testing/

SAST

- Security testing focus
  - SAST tools are tuned to find security defects
    - SQL injection, XSS, hardcoded credentials, weak crypto
    - Some tools also report code quality issues alongside security issues
  - SonarQube is a SAST too that
    - Does security scans
    - Identifies potential bugs
    - Identifies code smells
    - Provides metrics like cyclomatic complexity
  - Checkmarx is used by enterprises
    - More thorough but requires a licence

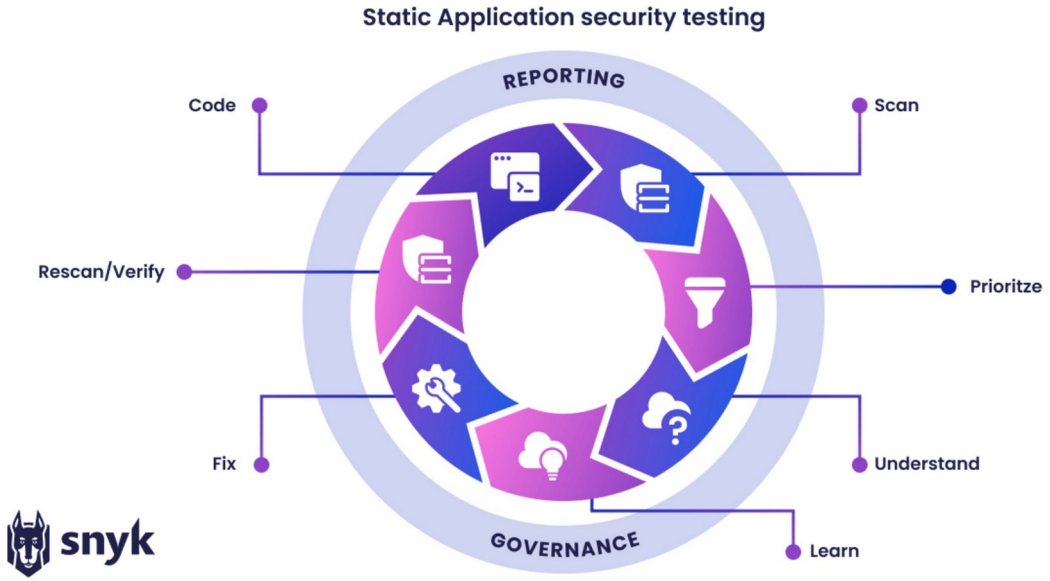


Image Credit:https://snyk.io/fr/articles/application-security/static-application-security-testing/

SAST

- How SAST works
  - Step 1: Parse
    - Read source code, build an Abstract Syntax Tree (AST)
  - Step 2: Model
    - Build a control flow graph showing how execution moves through the code
  - Step 3: Analyze
    - Perform taint analysis, tracking untrusted data from sources to sinks
  - Step 4: Report
    - Generate findings for code patterns that match known vulnerability rules

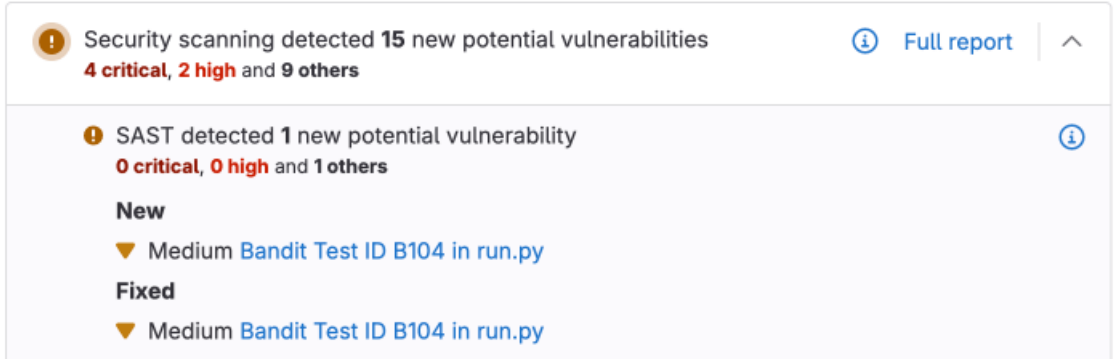


Image Credit: https://gitlab-docs-d6a9bb.gitlab.io/ee/user/application_security/sast/

SAST

- What SAST catches
  - Injection vulnerabilities
    - SQL injection, command injection, LDAP injection, XPath injection
  - Cross-site scripting (XSS)
    - Unescaped user input in HTML output
  - Hardcoded credentials
    - Passwords, API keys, tokens in source files
  - Weak cryptography
    - MD5/SHA-1 for security purposes, ECB mode, hardcoded keys, weak random number generation

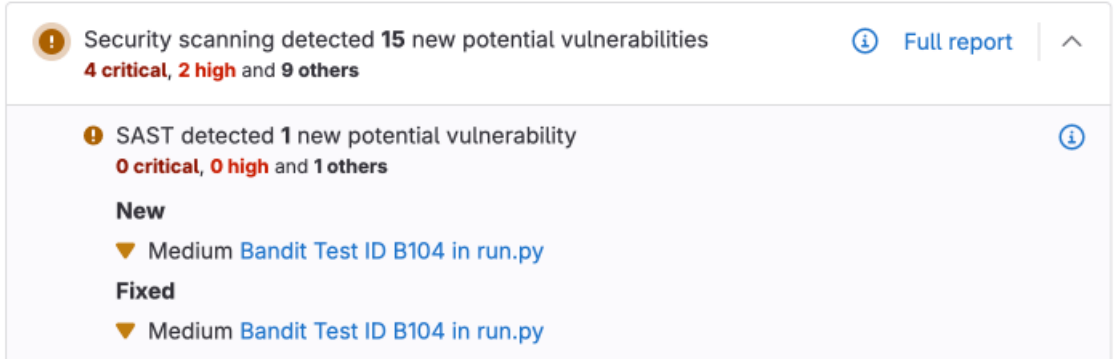


Image Credit: https://gitlab-docs-d6a9bb.gitlab.io/ee/user/application_security/sast/

SAST

- What SAST catches
  - Path traversal
    - User input controlling file paths
  - Insecure deserialization
    - Untrusted data passed to readObject(), Jackson, etc.
  - Information leakage
    - Logging sensitive data, exposing stack traces, debug info in responses

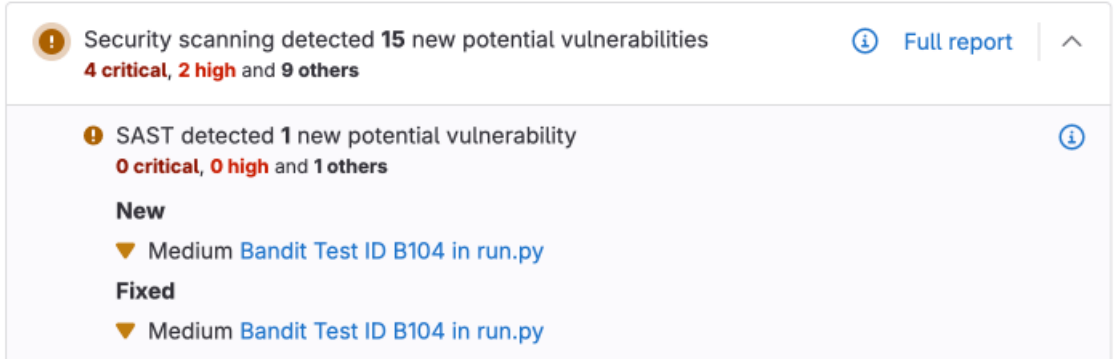


Image Credit: https://gitlab-docs-d6a9bb.gitlab.io/ee/user/application_security/sast/

SAST

- Injection
  - The canonical example
  - The pattern is always source-flows-to-sink-without-sanitizer
  - SAST tools detect it well
  - These all share the pattern
    - SQL injection (CWE-89)
    - Command injection (CWE-78)
    - LDAP injection (CWE-90)
    - XPath injection (CWE-91)
    - Header injection (CWE-93)

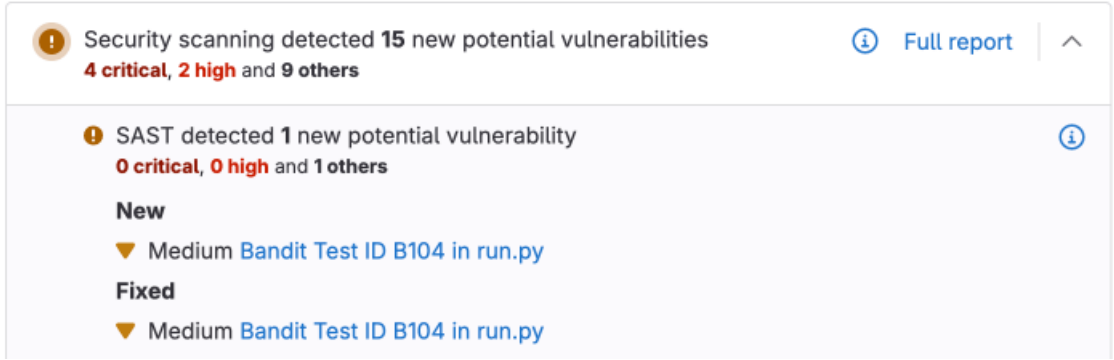


Image Credit: https://gitlab-docs-d6a9bb.gitlab.io/ee/user/application_security/sast/

SAST

- Hardcoded credentials
  - Detected by pattern matching
  - Looking for string literals that look like passwords or keys, often with regex against common variable names
  - Less sophisticated than taint analysis but quite effective
- Path traversal
  - Taint analysis applied to file system operations
    - User input flows to new File(...) or Path.of(...)
    - The analyzer asks whether the input was sanitized before use

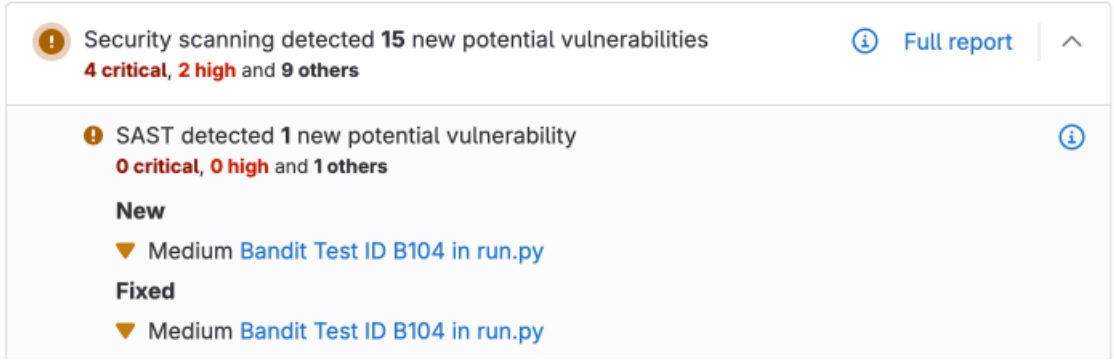


Image Credit: https://gitlab-docs-d6a9bb.gitlab.io/ee/user/application_security/sast/

SAST

- What SAST misses
  - Business logic flaws
    - "Users can transfer money from accounts they don't own"
  - Authentication misconfiguration
    - Security rules that aren't applied to the right URLs
  - Authorization bugs
    - Checking the wrong role for the wrong resource
  - Race conditions
    - Concurrency bugs that depend on timing

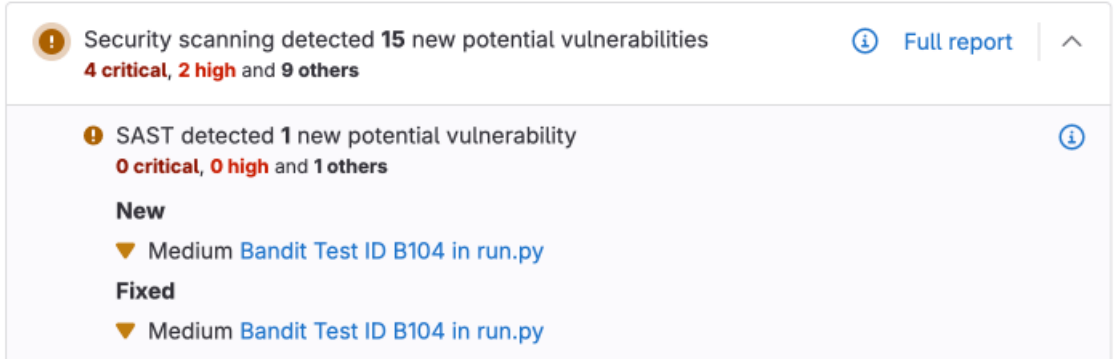


Image Credit: https://gitlab-docs-d6a9bb.gitlab.io/ee/user/application_security/sast/

SAST

- What SAST misses
  - Cryptographic protocol misuse
    - Using crypto correctly per API but wrong for the threat model
  - Information disclosure through observable behavior
    - Timing attacks, error message variation
  - Anything requiring runtime context
    - Environment variables, deployed configuration, user roles in production

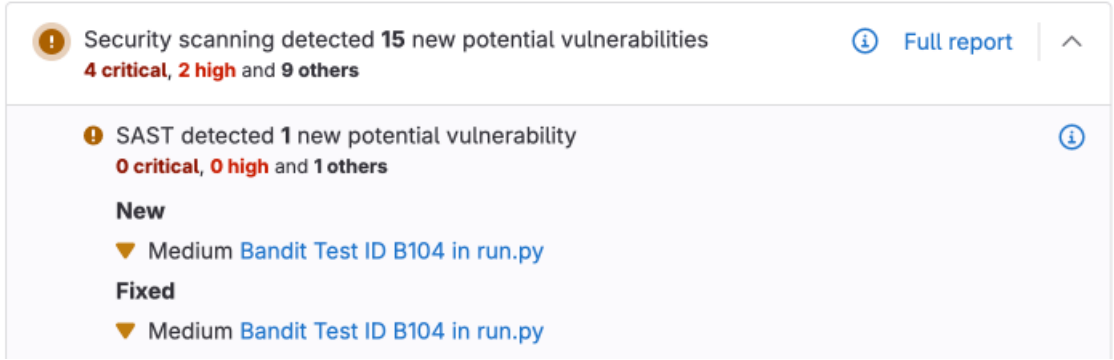


Image Credit: https://gitlab-docs-d6a9bb.gitlab.io/ee/user/application_security/sast/

CWE

- Common Weakness Enumeration (CWE)
  - CWE is a community-maintained taxonomy of software weaknesses
  - Every common vulnerability class has a CWE number
    - CWE-89 SQL Injection, CWE-79 XSS, CWE-798 Hardcoded Credentials, CWE-22 Path Traversal
  - Every modern SAST tool tags findings with CWE numbers
    - A finding tagged "CWE-89" means the same thing in Checkmarx, SonarQube, Snyk, Semgrep
  - The OWASP top 10 maps to CWE categories
    - The Top 10 is a curated list
    - The CWE is the vocabulary

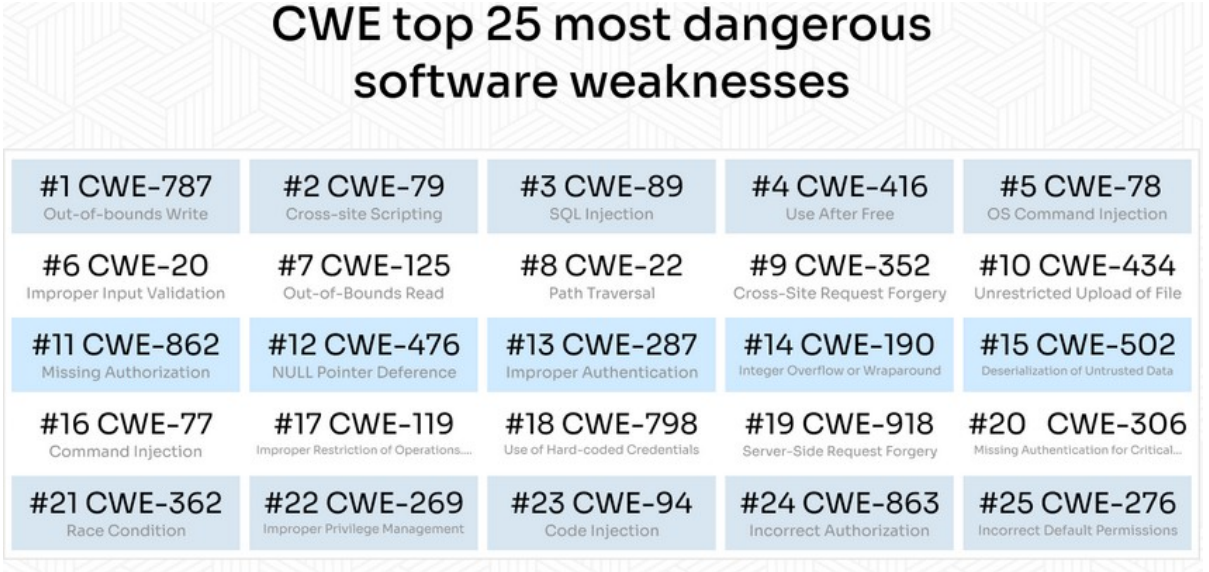


Image Credit: https://www.spinnakersupport.com/blog/2023/12/22/cve-vs-cwe/

CWE

- A few of the most common CWEs
  - CWE-89: SQL Injection
  - CWE-79: Cross-site Scripting (XSS)
  - CWE-78: OS Command Injection
  - CWE-22: Path Traversal
  - CWE-798: Use of Hard-coded Credentials
  - CWE-327: Use of a Broken or Risky Cryptographic Algorithm
  - CWE-352: Cross-Site Request Forgery (CSRF) -- this should ring a bell from Lab 4.8
  - CWE-502: Deserialization of Untrusted Data
  - CWE-200: Information Exposure

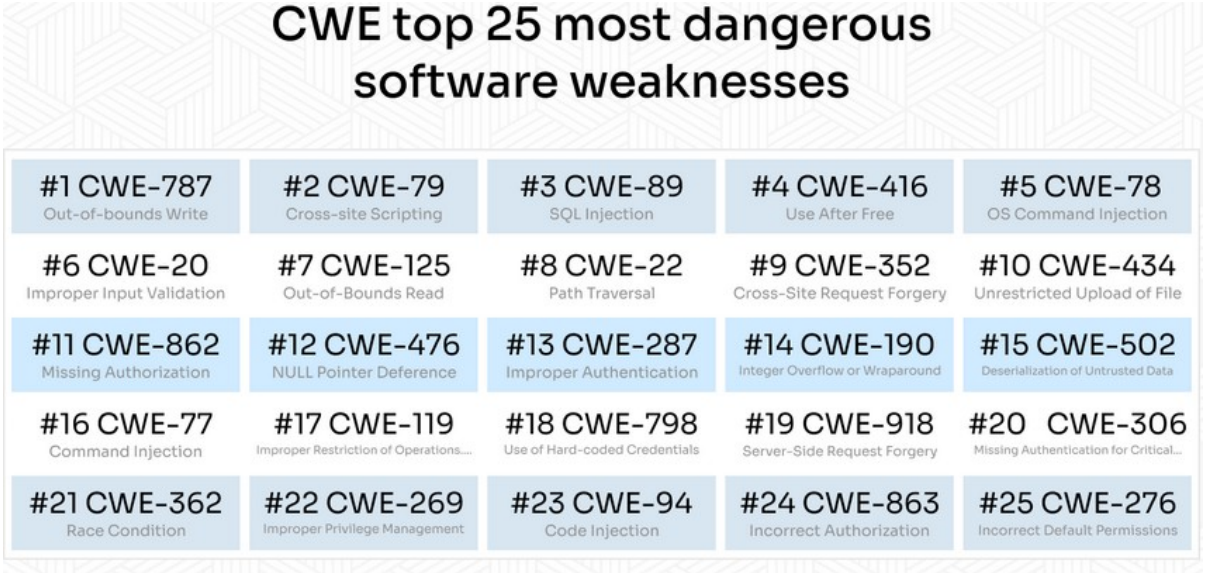


Image Credit: https://www.spinnakersupport.com/blog/2023/12/22/cve-vs-cwe/

False Positives

- False positives
  - Every SAST tool reports things that aren't necessarily vulnerabilities
  - Because
    - A SAST tool that finds nothing is broken
    - A SAST tool that finds everything is useless
  - The tool errs on the side of reporting, leaves judgment to humans
  - "False positive" means the tool was wrong about the data flow or the risk

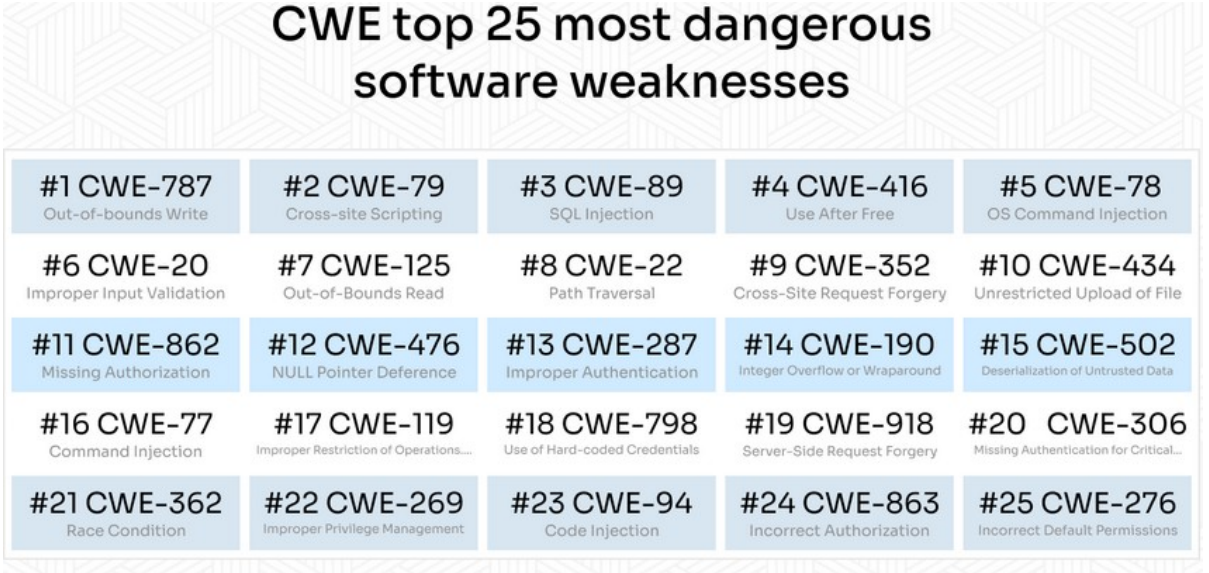


Image Credit: https://www.spinnakersupport.com/blog/2023/12/22/cve-vs-cwe/

False Positives

- False positives
  - Every SAST tool produces them
    - This is a deliberate design choice
    - The tool defers judgment to a human (you) for the cases where it can't be certain
  - The skill is distinguishing real findings from false positives
  - Two general mistake types:
    - *Accept everything*: treat every finding as real, fix or suppress all of them, burn out on the volume
    - *Dismiss everything*: assume the tool doesn't understand and dismiss findings without analysis, miss real issues

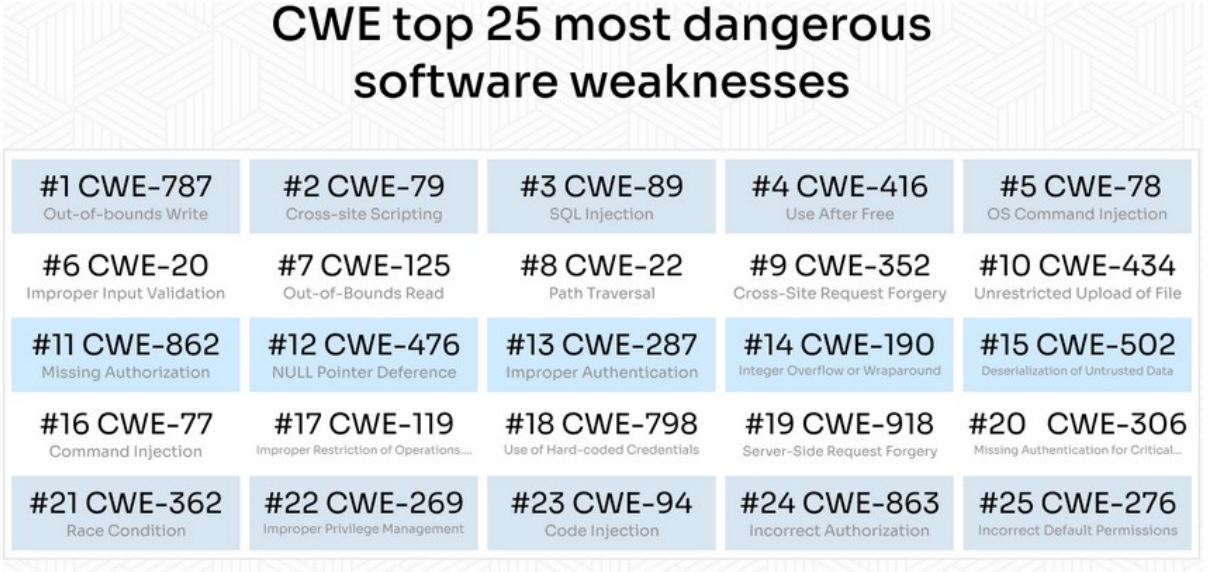


Image Credit: https://www.spinnakersupport.com/blog/2023/12/22/cve-vs-cwe/

Triage Workflow

- Confirm
  - Read the data flow trace the tool provides
    - Most SAST tools show the path from source to sink, line by line.
  - Walk through it mentally
    - Is the data actually untrusted?
    - Does it actually reach the sink?
    - Is there a sanitizer the tool missed?
  - At the end of this step, you have a position
    - Yes, this is real
    - Or no, the tool got something wrong.

```
For each finding:

  1. CONFIRM      -- Read the data flow. Does the alleged vulnerability exist?

  2. CLASSIFY     -- Pick one:
                     - True positive, fix now
                     - True positive, accepted risk (document)
                     - False positive (document why)
                     - Out of scope (test code, generated, etc.)

  3. ACT          -- Fix the code OR document the suppression OR confirm scope

  4. VERIFY       -- Re-scan to confirm fix or that suppression applied correctly
```

Triage Workflow

- Classify
  - Four categories cover almost every finding
  - True positive, fix now
    - The vulnerability is real, has no compensating control, should be remediated immediately
    - SQL injection in user-facing code is the canonical example.
  - True positive, accepted risk
    - The vulnerability is real but mitigated by other factors
    - The endpoint is internal-only and behind authentication
    - The system is air-gapped
    - The compensating control is documented and reviewed by security
    - The classification acknowledges the issue while explaining why immediate fix isn't required.

```
For each finding:

  1. CONFIRM      -- Read the data flow. Does the alleged vulnerability exist?

  2. CLASSIFY     -- Pick one:
                       - True positive, fix now
                       - True positive, accepted risk (document)
                       - False positive (document why)
                       - Out of scope (test code, generated, etc.)

  3. ACT          -- Fix the code OR document the suppression OR confirm scope

  4. VERIFY       -- Re-scan to confirm fix or that suppression applied correctly
```

Triage Workflow

- Classify
  - False positive: the tool is wrong
    - Either the data flow doesn't actually exist (sanitizer missed)
    - Or the alleged risk doesn't apply (the sink isn't actually sensitive)
    - Document why so a future developer or auditor needs to understand the reasoning
  - Out of scope:
    - The finding is in code that doesn't get deployed
    - Test code, generated code, third-party libraries
    - Often these can be excluded by configuration before scanning, but if they sneak in, classify them as out of scope with a note.

```
For each finding:

  1. CONFIRM      -- Read the data flow. Does the alleged vulnerability exist?

  2. CLASSIFY     -- Pick one:
                       - True positive, fix now
                       - True positive, accepted risk (document)
                       - False positive (document why)
                       - Out of scope (test code, generated, etc.)

  3. ACT          -- Fix the code OR document the suppression OR confirm scope

  4. VERIFY       -- Re-scan to confirm fix or that suppression applied correctly
```

Triage Workflow

- Act
  - For "fix now," change the code
  - For "accepted risk" and "false positive," create the suppression in the tool with the documented justification
  - For "out of scope," either fix the scan configuration or mark them in the tool.
- Verify
  - Always re-scan after fixing
    - The finding should be gone (or at least transformed into a different finding).
    - If it's still there, your fix didn't address what the tool was complaining about.

```
For each finding:

  1. CONFIRM      -- Read the data flow. Does the alleged vulnerability exist?

  2. CLASSIFY     -- Pick one:
                     - True positive, fix now
                     - True positive, accepted risk (document)
                     - False positive (document why)
                     - Out of scope (test code, generated, etc.)

  3. ACT          -- Fix the code OR document the suppression OR confirm scope

  4. VERIFY       -- Re-scan to confirm fix or that suppression applied correctly
```

Example

- Triage
  - Read the trace
    - Does the user-supplied accountId actually reach the SQL execution unsanitized?
  - Inspect the code.
    - If the query uses + concatenation: it's real
    - If it uses ? placeholders with setString: false positive, then the tool missed the parameterization
  - Decide
    - True positive, fix now
    - False positive, suppress with justification.
  - Re-scan
    - Confirm the finding is gone after the fix

```
[HIGH] CWE-89 SQL Injection in TransactionRepository.java:42

Source:  request.getParameter("accountId") at TransactionController.java:18
Path:    accountId → transactionService.getByAccount(accountId) → ...
Sink:    statement.executeQuery(query) at TransactionRepository.java:42

Suggested fix: Use parameterized queries.
```

Severity Levels and Prioritization

- Not all findings are equal
  - Critical / High
    - Direct exploitability, business impact
    - RCE, SQL injection, hardcoded prod credentials
  - Medium
    - Exploitable in some contexts
    - XSS in admin pages, weak crypto for non-secret data
  - Low
    - Defense-in-depth issues
    - Verbose error messages, deprecated APIs, missing security headers
  - Info
    - Code quality observations, not exploitable on their own
  - CI pipeline typically blocks merge on Critical/High; tracks Medium; informational for Low

```
[HIGH] CWE-89 SQL Injection in TransactionRepository.java:42

Source:  request.getParameter("accountId") at TransactionController.java:18
Path:    accountId → transactionService.getByAccount(accountId) → ...
Sink:    statement.executeQuery(query) at TransactionRepository.java:42

Suggested fix: Use parameterized queries.
```

Mapping SonarQube to Checkmarx

- Quick comparison
  - Cost
    - SonarQube Community is free and open source
    - Checkmarx is a commercial product with per-seat or per-scan licensing
  - Deployment
    - SonarQube Community runs locally in a Docker container
    - Checkmarx is typically a centrally-managed enterprise installation or SaaS subscription
  - Setup time for the labs
    - SonarQube takes minutes (one Docker command)
    - Checkmarx requires procurement, account provisioning, and security team coordination over weeks

Concept	SonarQube (lab)	Checkmarx (likely at the bank)
Severity	Blocker / Critical / Major / Minor / Info	Critical / High / Medium / Low / Information
Finding identity	Issue with key, rule, severity, location	Result with query name, severity, CWE
Triage status	Open / Confirmed / Resolved / Reopened	To Verify / Confirmed / Not Exploitable / Proposed Not Exploitable
False positive flag	Resolution: "False Positive" with comment	Mark as "Not Exploitable" with justification
Vulnerability vocab	CWE numbers	CWE numbers (same)
Scan trigger	Maven/Gradle plugin or CLI	CI integration, IDE plugin, CLI

Mapping SonarQube to Checkmarx

- Quick comparison
  - Java coverage
    - Both produce strong findings on Java code
    - The specific findings differ but the categories overlap substantially
  - Vulnerability vocabulary
    - Both use CWE numbers, so a finding's classification is portable between tools
  - Triage workflow
    - Both support the same workflow only the UI vocabulary differs
  - Banking adoption
    - Checkmarx is more common in large banks for the central CI pipeline
    - SonarQube is more common at the team/project level and for code quality analysis alongside security

Concept	SonarQube (lab)	Checkmarx (likely at the bank)
Severity	Blocker / Critical / Major / Minor / Info	Critical / High / Medium / Low / Information
Finding identity	Issue with key, rule, severity, location	Result with query name, severity, CWE
Triage status	Open / Confirmed / Resolved / Reopened	To Verify / Confirmed / Not Exploitable / Proposed Not Exploitable
False positive flag	Resolution: "False Positive" with comment	Mark as "Not Exploitable" with justification
Vulnerability vocab	CWE numbers	CWE numbers (same)
Scan trigger	Maven/Gradle plugin or CLI	CI integration, IDE plugin, CLI

Suppression Justifications

- Writing Justifications for audit
  - Bad:
    - False positive."
    - "I don't think this is a problem."
  - Better:
    - "False positive. The tool identified accountId flowing to executeQuery without sanitization, but inspection shows the JDBC PreparedStatement uses positional parameters (line 42, statement.setString(1, accountId)).
    - The tool's data flow analysis did not recognize the parameterization through the prepareStatement / setString idiom. No remediation required."
  - Best:
    - [the better text above] +
    - "Reviewed with [security team contact] on [date]. Filed as suppression #1234 with reference to internal AppSec wiki page https://..."

Concept	SonarQube (lab)	Checkmarx (likely at the bank)
Severity	Blocker / Critical / Major / Minor / Info	Critical / High / Medium / Low / Information
Finding identity	Issue with key, rule, severity, location	Result with query name, severity, CWE
Triage status	Open / Confirmed / Resolved / Reopened	To Verify / Confirmed / Not Exploitable / Proposed Not Exploitable
False positive flag	Resolution: "False Positive" with comment	Mark as "Not Exploitable" with justification
Vulnerability vocab	CWE numbers	CWE numbers (same)
Scan trigger	Maven/Gradle plugin or CLI	CI integration, IDE plugin, CLI

Suppression Justifications

- A bad justification ("false positive")
  - Tells the future reader nothing.
  - They have to redo your analysis from scratch, often without the context you had
  - They may incorrectly accept your dismissal
  - They may incorrectly reject it
  - Either way, you've left them worse off than necessary.

Concept	SonarQube (lab)	Checkmarx (likely at the bank)
Severity	Blocker / Critical / Major / Minor / Info	Critical / High / Medium / Low / Information
Finding identity	Issue with key, rule, severity, location	Result with query name, severity, CWE
Triage status	Open / Confirmed / Resolved / Reopened	To Verify / Confirmed / Not Exploitable / Proposed Not Exploitable
False positive flag	Resolution: "False Positive" with comment	Mark as "Not Exploitable" with justification
Vulnerability vocab	CWE numbers	CWE numbers (same)
Scan trigger	Maven/Gradle plugin or CLI	CI integration, IDE plugin, CLI

Suppression Justifications

- A good justification has three properties:
  - States the tool's allegation
    - "The tool identified X flowing to Y."
    - This shows you understood what the tool was reporting, not just that you disagreed.
  - Explains the actual code behavior
    - "Inspection shows the code uses parameterized queries via PreparedStatement.setString."
    - This is the technical fact that contradicts the tool's allegation.
  - Identifies WHY the tool was wrong
    - "The tool's data flow analysis did not recognize the JDBC parameterization idiom."
    - This is the diagnostic insight that explains the false positive at the level of the tool's behavior, not just the code.

Concept	SonarQube (lab)	Checkmarx (likely at the bank)
Severity	Blocker / Critical / Major / Minor / Info	Critical / High / Medium / Low / Information
Finding identity	Issue with key, rule, severity, location	Result with query name, severity, CWE
Triage status	Open / Confirmed / Resolved / Reopened	To Verify / Confirmed / Not Exploitable / Proposed Not Exploitable
False positive flag	Resolution: "False Positive" with comment	Mark as "Not Exploitable" with justification
Vulnerability vocab	CWE numbers	CWE numbers (same)
Scan trigger	Maven/Gradle plugin or CLI	CI integration, IDE plugin, CLI

Suppression Justifications

- The "Best" version
  - Adds review and traceability
    - A senior security person reviewed and concurred
    - A unique suppression ID in the tracking system links to the conversation
    - A wiki link or document reference provides extended context
    - This level of rigor is appropriate for high-severity findings or for codebases under regulatory scrutiny

Concept	SonarQube (lab)	Checkmarx (likely at the bank)
Severity	Blocker / Critical / Major / Minor / Info	Critical / High / Medium / Low / Information
Finding identity	Issue with key, rule, severity, location	Result with query name, severity, CWE
Triage status	Open / Confirmed / Resolved / Reopened	To Verify / Confirmed / Not Exploitable / Proposed Not Exploitable
False positive flag	Resolution: "False Positive" with comment	Mark as "Not Exploitable" with justification
Vulnerability vocab	CWE numbers	CWE numbers (same)
Scan trigger	Maven/Gradle plugin or CLI	CI integration, IDE plugin, CLI

Dynamic Security Application Testing

DAST

- Dynamic application security testing
  - Dynamic
    - Runs against the actual running application
  - Black-box
    - The scanner does not see the source code, only the application's externally observable behavior
  - Late in the SDLC
    - Typically runs against staging or pre-production environments
    - The application is started
    - The scanner crawls it and discovers endpoints
    - Then probes those endpoints with crafted attack payloads
    - Observes responses to determine whether the application is vulnerable

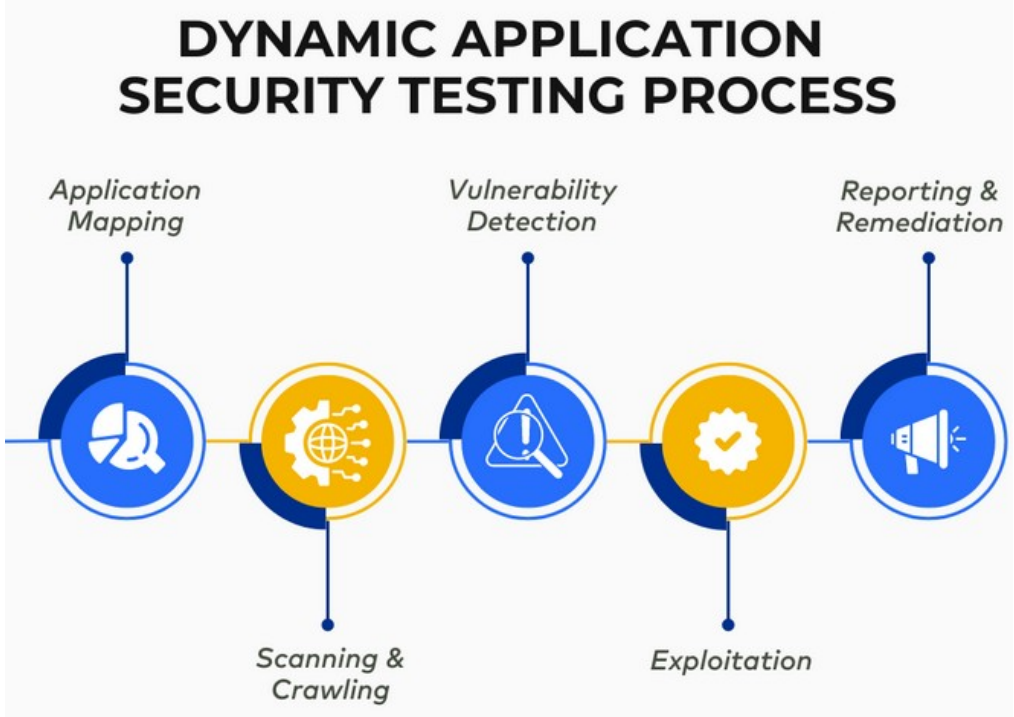


Image Credit: https://www.getastra.com/blog/dast/what-is-dast/

DAST Scan Lifecycle

- Lifecycle
  - Deploy
    - DAST requires a running application
    - Biggest practical constraint
    - Unlike SAST, DAST needs an environment to point at
    - Usually this is a staging environment, a deployed copy of the application that mirrors production.
  - Crawl
    - The scanner discovers what URLs the application has.
    - For traditional server-rendered apps with HTML links, this is straightforward
    - Follow every <a href>, submit every <form>
    - For modern SPAs like React crawling is harder because URLs are constructed by JavaScript at runtime.
    - Some DAST tools include a JavaScript engine to handle this
    - Others rely on developers providing a list of URLs to scan, often via a Postman/OpenAPI exports

```
1. DEPLOY     -- Application is running in staging environment
                 (real database, real config, real network)

2. CRAWL      -- Scanner discovers endpoints by following links,
                 submitting forms, exploring the URL space

3. PROBE      -- For each endpoint, send crafted attack payloads:
                 - "' OR 1=1--" for SQL injection
                 - "<script>alert(1)</script>" for XSS
                 - "../../../etc/passwd" for path traversal

4. OBSERVE    -- Watch responses for signs the attack succeeded:
                 - SQL errors in response
                 - Reflected payloads in HTML
                 - File contents leaking through

5. REPORT     -- Findings include the URL, the payload, the response,
                 and a CWE classification
```

DAST Scan Lifecycle

- Lifecycle
  - Probe
    - The scanner has a library of attack payloads tagged by vulnerability type
    - For each form field, query parameter, or header, it tries the relevant payloads
    - This is genuinely sending attack traffic to the application.
  - Observe
    - The scanner looks for signals that an attack worked
    - SQL injection signals: database error messages in the response.
    - XSS signals: the payload appearing in the response unescaped
    - Path traversal signals: file contents in the response
    - Each signal corresponds to a different finding type.

```
1. DEPLOY     -- Application is running in staging environment
                 (real database, real config, real network)

2. CRAWL      -- Scanner discovers endpoints by following links,
                 submitting forms, exploring the URL space

3. PROBE      -- For each endpoint, send crafted attack payloads:
                 - "' OR 1=1--" for SQL injection
                 - "<script>alert(1)</script>" for XSS
                 - "../../../etc/passwd" for path traversal

4. OBSERVE    -- Watch responses for signs the attack succeeded:
                 - SQL errors in response
                 - Reflected payloads in HTML
                 - File contents leaking through

5. REPORT     -- Findings include the URL, the payload, the response,
                 and a CWE classification
```

DAST Scan Lifecycle

- Lifecycle
  - Report
    - Findings include the URL, the payload that triggered the finding, the response, and a CWE classification
    - Same CWE vocabulary as SAST.
  - DAST scans are slow
    - A thorough DAST scan against a moderately-sized web application can take hours
    - They also generate noise in production logs
    - This is why DAST runs in staging, not production.

```
1. DEPLOY     -- Application is running in staging environment
                 (real database, real config, real network)

2. CRAWL      -- Scanner discovers endpoints by following links,
                 submitting forms, exploring the URL space

3. PROBE      -- For each endpoint, send crafted attack payloads:
                 - "' OR 1=1--" for SQL injection
                 - "<script>alert(1)</script>" for XSS
                 - "../../../etc/passwd" for path traversal

4. OBSERVE    -- Watch responses for signs the attack succeeded:
                 - SQL errors in response
                 - Reflected payloads in HTML
                 - File contents leaking through

5. REPORT     -- Findings include the URL, the payload, the response,
                 and a CWE classification
```

DAST and SAST

- What DAST does that SAST doesn't
  - Runtime configuration issues
    - Environment variables
    - Deployed application.yml values
    - Infrastructure settings
  - Deployment misconfigurations
    - Exposed admin endpoints
    - Debug mode enabled
    - Default credentials
  - Authentication flow problems
    - Session fixation
    - Token replay
    - password reset bypasses

```
1. DEPLOY     -- Application is running in staging environment
                 (real database, real config, real network)

2. CRAWL      -- Scanner discovers endpoints by following links,
                 submitting forms, exploring the URL space

3. PROBE      -- For each endpoint, send crafted attack payloads:
                 - "' OR 1=1--" for SQL injection
                 - "<script>alert(1)</script>" for XSS
                 - "../../../etc/passwd" for path traversal

4. OBSERVE    -- Watch responses for signs the attack succeeded:
                 - SQL errors in response
                 - Reflected payloads in HTML
                 - File contents leaking through

5. REPORT     -- Findings include the URL, the payload, the response,
                 and a CWE classification
```

DAST and SAST

- What DAST does that SAST doesn’t
  - Server-level vulnerabilities
    - Web server misconfigurations
    - Missing security headers
    - Weak TLS
  - Real-world exploitability
  - The SAST tool says "this could be exploited";
  - DAST proves "this WAS exploited"

```
1. DEPLOY     -- Application is running in staging environment
                 (real database, real config, real network)

2. CRAWL      -- Scanner discovers endpoints by following links,
                 submitting forms, exploring the URL space

3. PROBE      -- For each endpoint, send crafted attack payloads:
                 - "' OR 1=1--" for SQL injection
                 - "<script>alert(1)</script>" for XSS
                 - "../../../etc/passwd" for path traversal

4. OBSERVE    -- Watch responses for signs the attack succeeded:
                 - SQL errors in response
                 - Reflected payloads in HTML
                 - File contents leaking through

5. REPORT     -- Findings include the URL, the payload, the response,
                 and a CWE classification
```

DAST and SAST

- What SAST does that DAST doesn’t
  - Code paths the scanner cannot reach
    - Internal APIs
    - Admin endpoints
    - Code behind feature flags
    - Error paths
  - Code quality issues that aren't yet exploitable
    - Weak crypto in development
    - Hardcoded credentials in source
    - Deprecated APIs

```
1. DEPLOY     -- Application is running in staging environment
                 (real database, real config, real network)

2. CRAWL      -- Scanner discovers endpoints by following links,
                 submitting forms, exploring the URL space

3. PROBE      -- For each endpoint, send crafted attack payloads:
                 - "' OR 1=1--" for SQL injection
                 - "<script>alert(1)</script>" for XSS
                 - "../../../etc/passwd" for path traversal

4. OBSERVE    -- Watch responses for signs the attack succeeded:
                 - SQL errors in response
                 - Reflected payloads in HTML
                 - File contents leaking through

5. REPORT     -- Findings include the URL, the payload, the response,
                 and a CWE classification
```

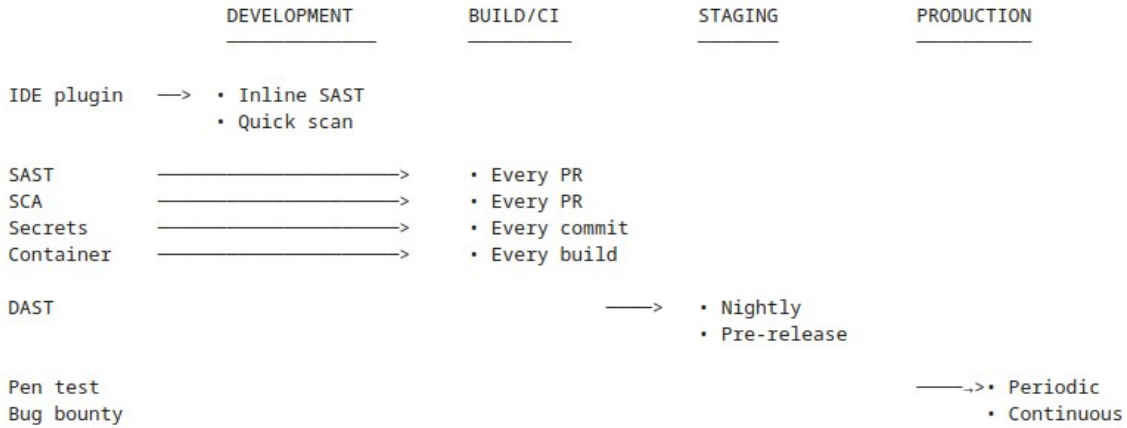
DAST and SAST

- What SAST does that DAST doesn’t
  - Vulnerabilities in
    - Non-exposed code
    - Internal microservice APIs
    - Batch jobs and scheduled tasks
  - Per-finding precision
    - SAST points at the exact line
    - DAST points at the URL
  - Coverage of cold paths
    - Code that runs rarely like error handlers, infrequent business cases
    - Gets analyzed by SAST regardless of whether it runs or not

```
1. DEPLOY     -- Application is running in staging environment
                 (real database, real config, real network)

2. CRAWL      -- Scanner discovers endpoints by following links,
                 submitting forms, exploring the URL space

3. PROBE      -- For each endpoint, send crafted attack payloads:
                 - "' OR 1=1--" for SQL injection
                 - "<script>alert(1)</script>" for XSS
                 - "../../../etc/passwd" for path traversal

4. OBSERVE    -- Watch responses for signs the attack succeeded:
                 - SQL errors in response
                 - Reflected payloads in HTML
                 - File contents leaking through

5. REPORT     -- Findings include the URL, the payload, the response,
                 and a CWE classification
```

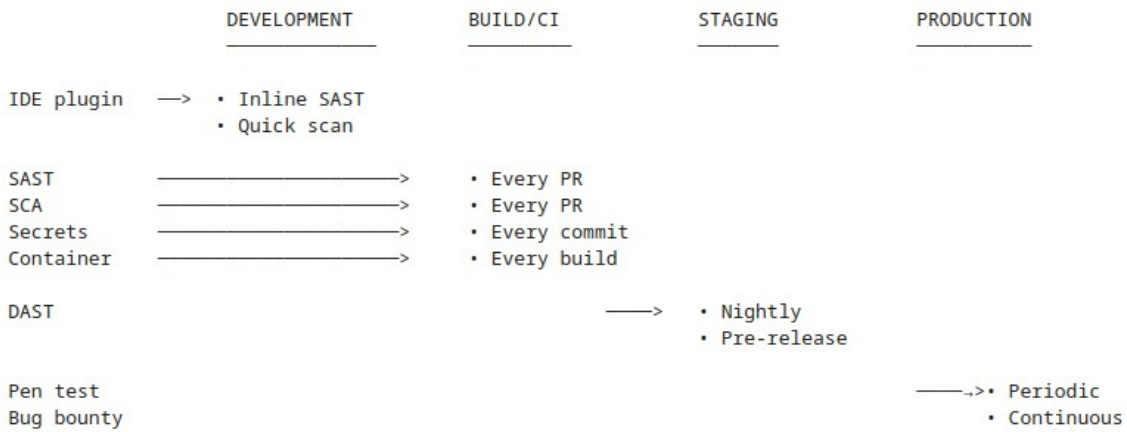
SDLC Fit

- For the bank context
  - All these layers exist in mature programs.
- Banks typically have
  - SAST and SCA in CI
    - For the bankbff, this would be triggered on every push
  - DAST runs against staging environments before production releases
  - Penetration tests at least annually, often more often for high-value applications
  - A bug bounty or vulnerability disclosure program for external reporting



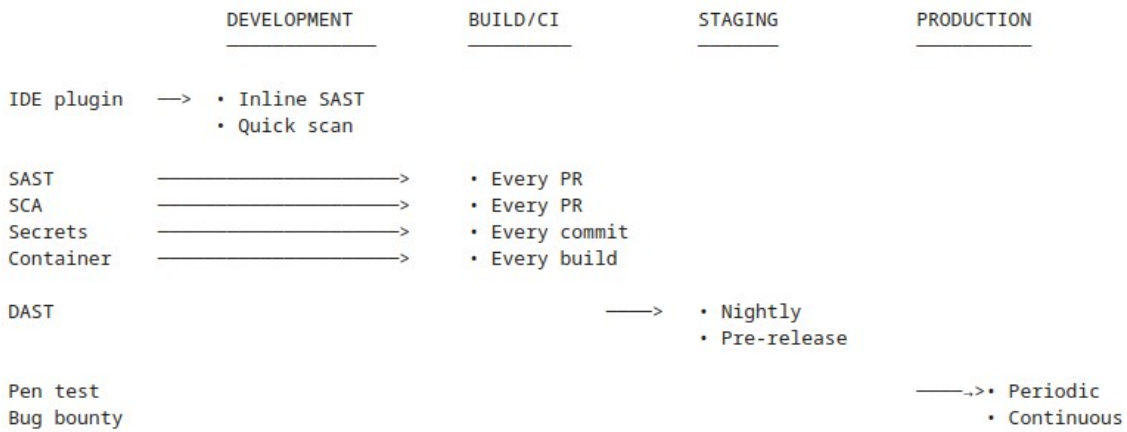
Other AppSec Layers

- Beyond SAST and DAST
  - SCA (Software Composition Analysis)
    - Scan dependencies for known vulnerabilities (CVEs)
  - Secrets scanning
    - Detect API keys, passwords, certificates accidentally committed to source control
  - Container scanning
    - Analyze Docker images for vulnerable OS packages and configurations
  - IAST (Interactive Application Security Testing)
    - Runtime instrumentation that combines SAST and DAST signals
  - Penetration testing
    - Humans, manually, attacking the application to find what scanners cannot
  - Bug bounty
    - External researchers paid to find and report vulnerabilities



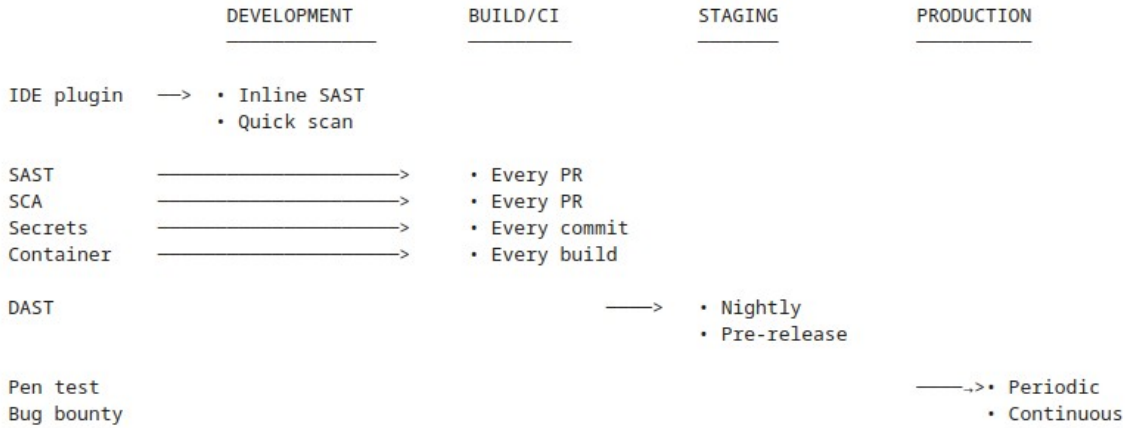
The Developer's Role

- You own
  - SAST findings
  - SCA findings
  - Secrets scanning findings
  - Fixing the vulnerabilities those tools surface
- You partially own
  - Unit and integration tests
  - Tests that catch security regressions



The Developer's Role

- Security engineers own
  - Running DAST scans
  - Configuring tooling
  - Maintaining baselines
  - Escalating findings
- Pen testers own
  - Finding what tools cannot find; their reports become work items for developers
- Bug bounty support own
  - Validating external reports
  - Valid findings become work items for developers



Questions?